Fractional ownership and borrowing in Lean

A design brief for the `Cap` subsystem

separate from, and not a claim of, the ITP manuscript

Gavin Mendel-Gleason

14 November 2026

Status. Implemented and building in this repository: `Cap/Id.lean`, `Cap/Share.lean`, `Cap/Footprint.lean`, `Cap/Admission.lean`, with accepted and refused controls under `bench/`. Proposed, not implemented: the context ledger, the typed core and its semantics, the elaborator, the interpreter and the backend refinement. Every Lean fragment below is quoted from the repository (or transcribed without change of content), and every output shown was observed at the revision this brief was generated from. Nothing here is a soundness result for the subsystem; the theorem families are listed in Section 8.

1 Why authority cannot live in a Lean value

Lean has no linear or affine typing. A value can be bound twice, and both names are equally usable. This is not a gap a careful wrapper closes: it is the host's semantics, and every decision below follows from it.

Listing 1: Duplication is legal, so a "unique" wrapper is not unique (probe form of `bench/OwnedDemo.lean`).

```
structure Owned (α : Type) where
  private mk ::
  val : α

def dup (a : Owned α) : Owned α × Owned α := (a, a)
```

For a mutable array the consequence is visible at run time: both copies can be written through, and the buffer's count is two, so the compiler copies on write rather than mutating. The type said "unique"; the runtime disagrees.

Listing 2: The restricted surface's positive and negative programs (`bench/OwnedAccept.lean`, `bench/OwnedReject.lean`).

```
def good : Owned Nat :=
  let a := new #[0, 0]
  let a := write a 0 1
  write a 1 2

def bad : Owned Nat :=
  let a := new #[0, 0]
  let _ := write a 0 1
  write a 1 2
```

```
#own_check good      -- accepted
#own_check bad
error: restricted ownership: handle 'a' used 2 times in its scope;
  the discipline permits one transfer
```

The subsystem therefore separates three things:

1. **Handles are names.** A `CapId` identifies a ledger entry; it carries no authority by itself. Forging a name is useless because the interpreter looks it up.
2. **Authority is a position in the current context.** A capability is *active*, *suspended*, *retired* or *consumed*; only the interpreter's context says which.
3. **The context is not a public value.** No `Ctx` → `Ctx` transition is exported, because such a function applied twice to one value would manufacture authority (Section 5).

2 Exact shares and the one-transfer rule

A context fixes a denominator D. A share is a positive numerator $q \le D$; the fraction is exactly q/D, with no rounding and no zero capability. A fractional split has children that add to the parent; a write needs the whole unit.

Listing 3: `Cap/Share.lean`.

```
def ShareWf (D q : Nat) : Prop := 0 < q ∧ q ≤ D
```

```
def Splits (q q1 q2 : Nat) : Prop := q1 + q2 = q

def Writable (D q : Nat) : Prop := q = D

theorem no_two_writes {D q q1 q2 : Nat} (h : Splits q q1 q2) (h1 : 0 < q1)
    (hw1 : Writable D q1) (hw2 : Writable D q2) (hq : q ≤ D) : False := by
  unfold Splits Writable at *
  omega
```

The theorem is the arithmetic content of the checker's "one transfer" rule: a unit context cannot split into two writable halves. A join is bounded (`join_le`), and splitting off the whole parent leaves nothing (`split_whole`).

A false lemma worth printing. The first version of the module claimed that the children's share bounds imply the parent's. It is false: two children each at most D sum to $2D$. `omega` produced the counterexample, and the claim was replaced by two monotonicity facts. The parent's $q \le D$ is a side condition of the split rule, not a consequence of the children.

Listing 4: The valid monotonicity facts; the parent-bound version was removed.

```
theorem split_le_parent_left {q q1 q2 : Nat} (h : Splits q q1 q2) : q1 ≤ q := by
  unfold Splits at h
  omega
```

Concrete checks that build with the module:

Listing 5: Decidable examples in `Cap/Share.lean`.

```
example : Splits 4 1 3 := rfl
example : ¬ Splits 4 3 2 := by unfold Splits; decide
example : Writable 8 8 := rfl
example : ¬ Writable 8 7 := by unfold Writable; decide
example : ShareWf 8 3 := by unfold ShareWf; decide
```

3 Bounded intervals and views

A footprint is half-open, with its ordering proof, and a view names an allocation and a footprint inside it. Splitting is where the arithmetic meets the geometry.

Listing 6: `Cap/Footprint.lean`.

```
structure Interval where
  lo : Nat
  hi : Nat
  ordered : lo ≤ hi

def Interval.Disjoint (a b : Interval) : Prop :=
  a.hi ≤ b.lo ∨ b.hi ≤ a.lo

def Interval.splitAt (i : Interval) (k : Nat) (h1 : i.lo ≤ k) (h2 : k ≤ i.hi) :
    Interval × Interval :=
  (⟨i.lo, k, h1⟩, ⟨k, i.hi, h2⟩)
```

The children are disjoint and cover the parent, and an allocation bound passes through:

Listing 7: The split's facts and the within-preservation proof.

```
theorem splitAt_disjoint ... :
    (splitAt i k h1 h2).1.Disjoint (splitAt i k h1 h2).2
theorem splitAt_adjacent ... :
    (splitAt i k h1 h2).1.Adjacent (splitAt i k h1 h2).2
theorem splitAt_cover ... :
    (splitAt i k h1 h2).1.lo = i.lo ∧ (splitAt i k h1 h2).2.hi = i.hi

theorem splitAt_within {len : Nat} {v : View} (hv : Within len v) (k : Nat)
    (h1 : v.range.lo ≤ k) (h2 : k ≤ v.range.hi) :
    Within len ⟨v.alloc, (splitAt v.range k h1 h2).1⟩ ∧
      Within len ⟨v.alloc, (splitAt v.range k h1 h2).2⟩ := by
  unfold Within Interval.splitAt at *
  simp_all
  omega
```

A view at `[0,64)` split at 32 gives disjoint children at `[0,32)` and `[32,64)`; splitting at 31 does not, and the module's example discharges that by `decide`.

4 Admitting a new value type

A type may be stored only if it is equivalent to an allowed representation, and the equivalence's two inverse laws make its shape a kernel-checked fact. The allowlist is closed on purpose.

Listing 8: `Cap/Admission.lean`: the allowlist and the certificate.

```
inductive SafeRep : Type → Prop where
  | nat : SafeRep Nat
  | bool : SafeRep Bool
  | uint8 : SafeRep UInt8
  | prod {A B : Type} : SafeRep A → SafeRep B → SafeRep (A × B)

class SafeValue (T : Type) where
  Rep : Type
  rep : SafeRep Rep
  toRep : T → Rep
  ofRep : Rep → T
  left : ∀ t, ofRep (toRep t) = t
  right : ∀ r, toRep (ofRep r) = r

structure SafeType where
  T : Type
  inst : SafeValue T
  new : Array UInt8 → T

def SafeType.init (st : SafeType) (bytes : Array UInt8) : st.T := st.new bytes
```

An admitted structure

Listing 9: `bench/AdmissionAccept.lean`: `Pair` is admitted through `Nat × Nat`.

```
structure Pair where
  x : Nat
  y : Nat

def Pair.new (_ : Array UInt8) : Pair := ⟨0, 0⟩

instance : SafeValue Pair where
  Rep := Nat × Nat
  rep := SafeRep.prod SafeRep.nat SafeRep.nat
  toRep p := (p.x, p.y)
  ofRep r := ⟨r.1, r.2⟩
  left := by intro p; cases p; rfl
  right := by intro r; cases r; rfl

#admit_type Pair
```

```
admitted 'Pair': a SafeValue instance was found
```

and the initialiser is applied to the allocation's bytes:

Listing 10: The allocator applies `new` against a fresh allocation's bytes.

```
def pairTy : SafeType := ⟨Pair, inferInstance, Pair.new⟩

example : pairTy.init #[1, 2] = ⟨0, 0⟩ := rfl
```

A refused structure

Listing 11: `bench/AdmissionReject.lean`: no `SafeRep` path to `IO.Ref`.

```
structure Bad where
  r : IO.Ref Nat

#admit_type Bad
```

```
error: failed to synthesize
  SafeValue Bad
```

Recursion through the parameters

Parameterised structures require the parameter's instance, so admission composes and refusal propagates.

Listing 12: `bench/AdmissionParam.lean`.

```
structure Wrap (α : Type) where
  val : α

instance [i : SafeValue α] : SafeValue (Wrap α) where
  Rep := i.Rep
  rep := i.rep
  toRep w := i.toRep w.val
  ofRep r := ⟨i.ofRep r⟩
  left := by intro w; cases w; rw [i.left _]
  right := by intro r; exact i.right r

structure Both (α β : Type) where
  fst : α
  snd : β

instance [i : SafeValue α] [j : SafeValue β] : SafeValue (Both α β) where
  Rep := i.Rep × j.Rep
  rep := SafeRep.prod i.rep j.rep
  toRep p := (i.toRep p.fst, j.toRep p.snd)
  ofRep r := ⟨i.ofRep r.1, j.ofRep r.2⟩
  left := by intro p; cases p; simp [i.left, j.left]
  right := by intro r; cases r; simp [i.right, j.right]
```

```
-- with SafeValue Nat, Bool and UInt8 as the base instances
example : SafeValue (Wrap Nat) := inferInstance
example : SafeValue (Wrap (Wrap Bool)) := inferInstance
example : SafeValue (Both Nat (Wrap UInt8)) := inferInstance

-- refusal propagates
error(lean.synthInstanceFailed): failed to synthesize instance of type class
  SafeValue (Wrap Bad)
```

The axiom forgery, and its refusal

An equivalence can be asserted rather than proved. The gate collects the instance's axioms and refuses anything outside `propext`, `Classical.choice` and `Quot.sound`.

Listing 13: `bench/AdmissionAxiomReject.lean`.

```
axiom badTo : Bad → Nat
axiom badFrom : Nat → Bad
axiom badLeft : ∀ b, badFrom (badTo b) = b
axiom badRight : ∀ n, badTo (badFrom n) = n

instance : SafeValue Bad where
  Rep := Nat
  rep := SafeRep.nat
  toRep := badTo
  ofRep := badFrom
  left := badLeft
  right := badRight

#admit_type Bad
```

```
error: the SafeValue instance for 'Bad' depends on non-standard axioms
  [badFrom, badLeft, badRight, badTo]
```

What the obligations do and do not say

Obligation	Meaning
`Rep`, `rep`	the representation is in the allowlist
`toRep`, `ofRep`	maps in both directions
`left`, `right`	the two inverse laws
`new : Array UInt8 → T`	initialiser used by allocation
axiom audit	the instance rests on no non-standard axiom

These are *shape* obligations. They certify that the type is bijective with an allowed representation; they do not state that the type itself carries no live resource. A bijection to `Nat` asserted by axioms is caught, but a genuine bijection is only a practical guard. Closing the gap needs either a definitional representation (admitted

types are aliases of the safe representation) or a handler that builds the instance from the type's declaration, with the faithfulness of that structural reading named as a trust assumption.

A catalogue of refused attempts

Attempt	Required rejection
duplicate one handle, join the aliases	both names are one consumed ledger entry
join fractions of different allocations	allocation-identity mismatch
join equal-length, different intervals	footprint mismatch
write through a half-share reader	insufficient write authority
rejoin a read-only loan, then write	read-only ceiling
split a writer, then use the old parent	suspended or consumed authority
borrow a slice, then free the parent	outstanding loan
end an outer lifetime before an inner loan	dependency violation
reuse an old handle after free and reallocation	stale identity or generation
capture a borrower in a returned closure	unsupported resource-bearing result
discard a reader in the linear core	explicit return or retirement required
retire a reader, then access it	retired claim
duplicate a saved close action in one session	not composable from the post-close context
free an empty slice	missing root disposal authority
lose a capability in one branch	explicit sum or common post-context
throw while a borrow is live	rejected until cleanup is specified
lift arbitrary IO into a borrowed scope	no such lifting operation

Automation. Plain structures are mechanical: representation as a product, componentwise maps, both laws by `cases/rfl`. Parameters are premises. The following are not automatic and should fail with an unresolved obligation rather than a silent assumption: types with invariants (`Fin n`, `Vector` α `n`), normalising types (sign–magnitude integers, rationals), quotients, opaque or privately constructed types, dependent fields, and inductive containers, which need a `SafeRep` extension before any handler can help. A type that cannot be admitted is simply not storable: that is a usability limit, not an unsoundness.

5 Allocation, freshness and the seal

Allocation is an instruction of the object language, not a function the client calls on a context. The arena issues the identifier, the admitted type's initialiser supplies the value, and the ledger records them together.

Listing 14: Allocation sketch. `next` is the arena's fresh counter; no client-supplied id appears.

```
structure Entry where
  id : AllocId
  T : Type
  inst : SafeValue T
  value : T

def alloc (next : Nat) (st : SafeType) (bytes : Array UInt8) : Entry :=
  ⟨⟨next⟩, st.T, st.inst, st.init bytes⟩
```

Four rules make the discipline unforgeable, and they are architectural rather than type-level:

1. The ledger is never a public value. No `Ctx` → `Ctx` transition is exported: a private constructor stops construction, not duplication, so a public split function applied twice to one value would mint authority.
2. The ledger and the runner live in one module, with the ledger's definitions `private`; Lean privacy is file-scoped, so no importing module can name them, and the runner's recursion is the only thread through the ledger.
3. The public boundary is a checked program description with a private constructor, carrying a kernel-checked typing derivation. A fabricated derivation fails elaboration; the checker is untrusted but cannot forge the kernel's acceptance.
4. A phantom arena parameter makes identifiers from one run ill-typed in another, and no runner executes a saved action against an existing arena: each run starts with fresh storage and a fresh identity namespace.

6 The reflective usage checker, and why it is not the seal

Before the ledger existed, a static prototype checked a restricted surface by reflection. It is a useful diagnostic and a useful negative result.

Listing 15: `bench/OwnedStatic.lean`: private payload, one-transfer check.

```
structure Owned (α : Type) where
  private mk ::
  private val : Array α

def new (xs : Array α) : Owned α := ⟨xs⟩
def write (o : Owned α) (i : Nat) (v : α) : Owned α := ⟨o.val.set! i v⟩
```

The check reflects the elaborated definition, counts uses of each `Owned`-typed binding by de Bruijn index, and refuses a second use.

```
-- accepted
#own_check good

-- refused
error: restricted ownership: handle 'a' used 2 times in its scope;
  the discipline permits one transfer

-- direct payload access
error: Field 'val' from structure 'Owned' is private
```

It cannot be the trust boundary: it reads a term and throws, producing no kernel-checked certificate, and it sees usage only where the shape is syntactic. The runtime-guarded counterpart is honest about the same limit:

```
-- bench/OwnedDemo.lean, a shared used-flag
consumed handle refused: owned handle used twice
duplicate refused: owned handle used twice
```

The runtime guard the prototype contrasts with is the guarded interface in `bench/OwnedDemo.lean`: a handle carries a shared used-flag, so a copy shares the flag and the second use fails at run time.

Listing 16: `bench/OwnedDemo.lean`: dynamic enforcement.

```
structure Owned (α : Type) where
  private mk ::
  val : α
  used : IO.Ref Bool

def new (a : α) : IO (Owned α) := do
  let r ← IO.mkRef false
  pure ⟨a, r⟩

def take (o : Owned α) : IO α := do
  if ← o.used.get then
    throw (IO.userError "owned handle used twice")
  o.used.set true
  pure o.val
```

7 The ledger design

The proposed context is a finite map of capabilities, a decomposition forest, a lifetime forest and loan records. Capability states are *active*, *suspended*, *retired* and *consumed*; a suspended parent is bookkeeping, not a second active budget.

For each allocated byte x, the terminal holders' shares must sum to one:

$$\mathrm{Budget}_\Gamma(a, x) = \sum_{\substack{c \in \mathrm{Frontier}(\Gamma) \\ c.a = a,\ x \in c.F}} c.q = 1.$$

Spatial splits preserve the equation because each byte is in exactly one child; fractional splits preserve it because the children sum to the parent; retirement keeps the contribution but removes access. The equation is necessary and not sufficient: provenance, ceiling inheritance, lifetime liveness, restoration removing descendants, a single disposal entitlement, and freshness across branches are separate conjuncts of the invariant.

Transition	Required effect
`alloc`	fresh allocation, root capability, disposal entitlement, zeroed bytes
`free`	root view, disposal entitlement, no fragments, no loans
`read`	active claim, live lifetime, positive share, in footprint
`write`	as read, plus whole share and read–write ceiling
`splitFraction`	children positive, sum to parent, inherit ceiling and lifetime
`joinFraction`	distinct claims, same allocation, footprint, lifetime, provenance
`splitAt`	in-bounds point, two children at the parent's share
`joinSpatial`	adjacency, coverage, matching shares and ceilings, no subloans
`beginBorrow`	parent suspended, loan with fresh lifetime, borrowed child
`endBorrow`	returned authority complete, nested loans ended, fresh restoration

A worked trace: 64 bytes, split, borrowed, restored

The intended surface program below is the design's acceptance example: allocate 64 bytes, take a mutable loan of the whole array, split it in two, fill both slices, split the right slice fractionally into two readers, scan each, rejoin the readers, rejoin the slices, return the loan, snapshot and free.

Step	Usable authority
allocate	owned root over `[0,64)` and its disposal entitlement
begin mutable scope	borrowed full writer over `[0,64)`; lender suspended
spatial split at 32	full writers over `[0,32)` and `[32,64)`
fractional split of right	left writer; two half-share readers over `[32,64)`
rejoin readers	left writer; restored full right writer
rejoin slices	full borrowed writer over `[0,64)`
end mutable scope	restored owned root and disposal entitlement
snapshot	same owned root; independent immutable bytes
free	no allocation or access authority remains

A retained alias `oldReader = reader1` still names the consumed capability after the join; a later read through it is rejected because the ledger entry is consumed, not because the source name changed.

The indexed judgement threads the context and a result-dependent post-context:

Listing 17: The design's computation type, schematic.

```
OwnM : (ρ : Type) → Ctx ρ → {α : Type} → (α → Ctx ρ) → Type

ret  : (x : α) → OwnM ρ Γ (fun _ => Γ)
bind : OwnM ρ Γ Q → ((x : α) → OwnM ρ (Q x) R) → OwnM ρ Γ R
```

Indices are hidden by elaboration, not discarded before composition: a `Sigma` Γ_{in}, `Sigma` Γ_{out}, `Program` Γ_{in} Γ_{out} α package cannot supply a generally valid bind unless the matching intermediate contexts are established.

8 Theorem obligations and gates

Theorem	Content
share arithmetic	executable shares agree with the rational model
context well-formedness	names, footprints, roots, ceilings, lifetimes
primitive preservation	each legal primitive preserves `Realizes`
bounds safety	every executed access is in its allocation and footprint
access exclusion	a writable byte has no competing budget outside the path
lifetime validity	usable claims belong to live loans with valid dependencies
reconstruction	joins and loan ends remove descendants before restoring
disposal safety	free consumes the unique entitlement, no live dependents
frame preservation	unaffected observations and authority are preserved
typed execution safety	a certified program cannot reach an ownership fault
scope confinement	scoped authority is not executable outside its boundary
cleanup	a closed successful program leaves no live allocation or loan
checker soundness	a successful check yields the declarative judgement
elaboration correctness	the certificate corresponds to the accepted source
backend refinement	mutable execution simulates the reference transitions

Stages, in dependency order: (1) shares, identities, intervals; (2) context ledger, forest, primitive transitions; (3) byte-array semantics; (4) fractional and spatial views; (5) explicit loans and reborrowing; (6) typed programs and the closed runner; (7) hidden-index elaborator; (8) optional retirement and scope recovery; (9) mutable interpreter; (10) specialisation and integrations. The first useful milestone is stage 4; the first programmer-facing one is stage 7.

9 What is deliberately not claimed

The subsystem is an interpreter with a checked access discipline over its own arena. It does not retrofit linearity onto Lean, does not make the host's values affine, and does not claim that Lean's reference counts reach one at a write site. Logical exclusive access is not a runtime representation property. Permission soundness, backend refinement and runtime uniqueness are separate theorem families, and only the first is close to being stated in the repository.

10 How to run the examples

From the repository root, with the toolchain from the dev shell:

```
nix develop -c lake build Cap

# admission controls (the refusals are intended)
cd bench
export LEAN_PATH=../.lake/build/lib/lean:$LEAN_PATH
lean AdmissionAccept.lean        # admitted 'Pair'
lean AdmissionReject.lean        # failed to synthesize SafeValue Bad
lean AdmissionParam.lean         # Wrap/Both instances
lean AdmissionParamReject.lean   # failed for Wrap Bad
lean AdmissionAxiomReject.lean   # non-standard axioms

# reflective prototype
lean OwnedAccept.lean            # accepted
lean OwnedReject.lean            # one-transfer violation
lean OwnedNoAccess.lean          # private payload
lean OwnedDemo.lean              # runtime guard
```

The full gate, including the paper and the certified fragment:

```
nix develop -c bash scripts/check_all.sh
```

A File map

File	Role
`Cap/Id.lean`	allocation, capability, loan, lifetime identities
`Cap/Share.lean`	shares, splits, the one-transfer theorem
`Cap/Footprint.lean`	intervals, views, split facts
`Cap/Admission.lean`	`SafeRep`, `SafeValue`, `SafeType`, the gate
`bench/Admission*.lean`	accepted, refused, recursive and axiomatic controls
`bench/Owned*.lean`	reflective and runtime prototypes
`docs/lean-fractional-ownership-design.md`	the full design note